

Bryck API Operations Manual

Complete User Guide for Storage Management

Bryck Engineering Team

August 2026

Contents

Bryck API Operations Guide — Comprehensive User Manual	1
0. Getting Started Guide — First-Time Setup	3
0.1 Quick Reference Card — Most Common Operations	10
1. Runner Reference — Organized by Category	13
2. Configuration files	18
3. Bryck state machine	24
4. Typical lifecycles	25
5. CLI	26
6. Exit codes	44
7. Key-file transfer detail	45
8. Expanded Troubleshooting Guide	47
9. Performance & Tuning Guide	49
10. Security & Credentials Best Practices	51
11. Examples Gallery — Real-World Scenarios	53
12. Technical Specifications	57
9. SSH transport (<code>ssh_runner.py</code>)	61

Bryck API Operations Guide — Comprehensive User Manual

Project release: `bryckclient-cli/v1.0.0`

This manual documents the standalone operator-side CLI release. It is kept separate from the repository's `dev_main` branch and may be integrated there later through a separate reviewed merge.

This is the **complete user manual** for operating a Bryck NVMe storage appliance from a remote Linux machine. It contains everything needed to format, mount, manage, and troubleshoot your Bryck from day one.

This guide covers: - First-time setup (credentials, configuration) - Step-by-step workflows for common operations - Full technical reference for all runners (35+ tools) - Troubleshooting and error resolution - Cloud operations (AWS/GCP/Azure transfers, notifications) - Performance tuning and security best practices

Quick navigation:

- **New user?** Start at *Getting Started Guide* (Section 0)
- **Need a quick example?** Jump to *Quick Reference Card* (Section 0.1)
- **Specific runner?** Find it in *Runner Reference* (Section 1) — organized by category
- **Something broken?** See *Troubleshooting Guide* (Section 8)

Quick Navigation

Section	Purpose
0 Getting Started	First-time setup guide
0.1 Quick Reference	Copy-paste commands
0.2 Common Mistakes	Common pitfalls and fixes
0.3 Validate Config	Verify login.json
1 Runner Reference	All 35+ runners
1.1 Lifecycle	scan, format, mount, eject, erase, remove
1.2 Information	info, network_info, report
1.3 System Config	change_ip, change_time
1.4 Cloud Config	cloud_configure, cloud_show, cloud_deconfigure
1.5 Cloud Transfer	initiate, status, pause, resume, cancel
1.6 Notifications	Email/SNS setup & alerts
2 Configuration Files	login.json, format_mount_params.json, etc.
3-4 Architecture	State machine & lifecycle
3 State Machine	Bryck state transitions
4 Lifecycle	Complete operational flow
5-7 Reference	Detailed documentation
5 Runner Docs	Full runner reference
6 Exit Codes	Exit code meanings
7 Key-file Transfer	Encryption key transfer
8-12 Troubleshooting	Diagnostics & specs
8 Troubleshooting	Symptoms & diagnosis
9 Performance	Tuning & optimization
10 Security	Credentials & security
11 Examples	Real-world scenarios
12 Specs	Technical specifications

0. Getting Started Guide — First-Time Setup

This section is for users running these runners for the first time.

0.0 System Requirements

On your operator machine (where you'll run the runners): - Python 3.7 or higher - Linux, macOS, or Windows with WSL2 - Network access to your Bryck (same network or routable) - Internet access to install Python packages

On your Bryck appliance: - Firmware v1.0 or later - REST API enabled (default) - SSH enabled (required for format/mount/erase operations)

Installation:

```
# Verify Python is installed
python3 --version # Should show 3.7+

# Install required packages
pip3 install requests paramiko

# macOS users (with Homebrew)
brew install python3
pip3 install requests paramiko

# Ubuntu/Debian users
sudo apt-get install python3 python3-pip
pip3 install requests paramiko

# RHEL/CentOS users
sudo yum install python3 python3-pip
pip3 install requests paramiko
```

0.1 Step-by-Step: First-Time Setup

Step 1: Locate Your Bryck's IP Address Find your Bryck's management IP address: - **On the Bryck itself:** Check the LCD display on the front panel - **From your network:** SSH to your network gateway and run `arp-scan` or check DHCP client table - **Expected format:** IPv4 address like 192.168.6.35 (NOT a hostname)

Example:

```
# Scan for Bryck on network
ping 192.168.6.35 # Should respond if Bryck is alive
```

Step 2: Create and Configure login.json This file stores your authentication credentials.

```
cd bryckclient-cli/  
  
# Copy the template  
cp login.example.json login.json  
  
# Edit with your favorite editor  
nano login.json  
# or  
vim login.json  
# or  
code login.json
```

Fill in these fields:

```
{  
  "bryckapi_host": "192.168.6.35",  
  "bryckapi_scheme": "http",  
  "bryckapi_port": "80",  
  "bryckapi_username": "admin",  
  "bryckapi_password": "your_password",  
  "timeout": 300,  
  "bryckserver_username": "bryck",  
  "bryckserver_password": "ssh_password"  
}
```

← Your Bryck IP (REQUIRED, IPv4 only)
← "http" or "https" (usually "http")
← Port number (usually 80 for http, 443 for https)
← REST API username (HINT: same as the default)
← REST API password (HINT: same as the default)
← Request timeout in seconds (300 = 5 minutes)
← SSH username (for format/mount/erase)
← SSH password (REQUIRED for format/mount)

Getting these values:

Field	Where to find	Example
bryckapi_host	Bryck LCD display or DHCP logs. MUST be IPv4, not hostname.	192.168.6.35
bryckapi_port	Ask your Bryck admin. Defaults: 80 (HTTP), 443 (HTTPS)	80
bryckapi_username / password	Bryck admin. Usually admin / factory password.	admin / pass123
bryckserver_username / password	SSH credentials. Usually bryck / factory password.	bryck / sshpass456
timeout	Leave as 300 unless operations are timing out	300

Example (working setup):

```
{  
  "bryckapi_host": "192.168.6.35",  
  "bryckapi_scheme": "http",  
  "bryckapi_port": "80",  
  "bryckapi_username": "admin",  
  "bryckapi_password": "your_password",  
  "timeout": 300,  
  "bryckserver_username": "bryck",  
  "bryckserver_password": "ssh_password"
```

```

    "bryckapi_port": "80",
    "bryckapi_username": "admin",
    "bryckapi_password": "bryckadmin123",
    "timeout": 300,
    "bryckserver_username": "bryck",
    "bryckserver_password": "bryck456"
}

```

Security tip: Never commit `login.json` to version control. Add it to `.gitignore`:

```
echo "login.json" >> .gitignore
```

Step 3: Test Connectivity Verify your setup works:

```
python3 bryck_info.py
```

Expected output (success):

```

{
  "bryck_info": {
    "State": " Removed",
    "Capacity": "4.0 TB",
    "Firmware": "v1.2.3",
    "Health": "OK"
  }
}

```

If you get an error:

Error Message	Cause	Fix
Connection refused	IP address wrong or Bryck offline	Check IP with <code>ping</code> <code>192.168.6.35</code>
Unauthorized (401)	Password wrong	Verify <code>bryckapi_password</code> in <code>login.json</code>
Hostname not recognized	Using hostname instead of IP	Use IPv4 only (e.g., <code>192.168.6.35</code>)
Timeout	Network unreachable	Check firewall, routing, network connectivity

Error Message	Cause	Fix
ssl: CERTIFICATE_VERIFY_FAILED	HTTPS with self-signed cert	Add <code>--verify False</code> (handled automatically)

Step 4: Create `format_mount_params.json` This file configures how to format and mount your Bryck.

```
# Copy and backup the template
cp format_mount_params.json format_mount_params.json.backup
nano format_mount_params.json
```

For first-time setup, use these defaults:

```
{
  "format": {
    "key_file": "",
    "encryption_option": "",
    "IoSize": "256",
    "DataSync": "application sync",
    "raid_level": 5,
    "filesystem": "zfs",
    "num_vols": "None"
  },
  "mount": {
    "key_file": "",
    "encryption_option": "",
    "mountonreboot": true,
    "force_check": false
  }
}
```

Field explanations:

Format section (for `bryck_format.py`):

Field	Meaning	Value	Notes
<code>key_file</code>	Encryption key file path (local path on your machine).	<code>""</code> — no encryption (default); <code>/path/to/key.key</code> — encrypted format	Leave empty for unencrypted Bryck. Provide a local file path only when formatting with encryption.
<code>encryption_option</code>	Encryption mode selector.	Always <code>""</code>	Must always be left empty. Do not change this field.
<code>IoSize</code>	Block size in KB. 256 = standard, balances perf & compatibility	<code>"256"</code>	Don't change unless advised by Bryck support
<code>DataSync</code>	Sync mode. <code>"application sync"</code> = app controls when data hits disk	<code>"application sync"</code>	Standard for most workloads
<code>raid_level</code>	RAID level: 1 (mirroring), 5 (striping+parity), 6 (dual parity)	<code>5</code>	5 balances performance & protection. Use 6 for critical data.
<code>filesystem</code>	Filesystem type: <code>zfs</code> (recommended) or <code>ext4</code>	<code>"zfs"</code>	ZFS has better protection & performance. Use <code>ext4</code> only if needed.
<code>num_vols</code>	Number of volumes. <code>"None"</code> = single volume (simplest)	<code>"None"</code>	Leave as "None" unless you need multiple volumes

Mount section (for `bryck_mount.py`):

Field	Meaning	Value	Notes
<code>key_file</code>	Encryption key file path. Must match the <code>key_file</code> used at format time.	<code>""</code> — no encryption (default); <code>/path/to/key.key</code> — encrypted mount	Leave empty if the Bryck was formatted without encryption.
<code>encryption_option</code>	Encryption mode selector.	Always <code>""</code>	Must always be left empty. Do not change this field.
<code>mountonreboot</code>	Remount after reboot? <code>true</code> = yes, <code>false</code> = no	<code>true</code>	Set to <code>true</code> so Bryck remounts automatically
<code>force_check</code>	Run filesystem check on mount?	<code>false</code>	Set to <code>false</code> for new format, <code>true</code> if you suspect corruption

Step 5: Create cloud_ops.json (For Cloud Transfers) Skip this step if you're not using cloud transfers. If you plan to upload/download data to/from AWS, GCP, or Azure, you'll need this file.

```
# Copy the example template
cp cloud_ops.example.json cloud_ops.json

# Edit with your cloud credentials
nano cloud_ops.json
```

Fill in these fields for AWS:

```
{
  "cloud_type": "aws",
  "access_key_id": "AKIAIOSFODNN7EXAMPLE",
  "secret_access_key": "wJalrXUtnFEMI/...",
  "region": "us-east-1",
  "keyfile": "",
  "tenant_id": "",
  "bryck_src": "/bryck/source-directory",
  "cloud_bucket": "s3://your-bucket/path",
  "bryck_dst": "/bryck/download-directory",
  "notification": {
    "sns_topic": "arn:aws:sns:...:your-topic",
    "sqs_queue": "arn:aws:sqs:...:your-queue",
    "emails": ["you@example.com"],
    "states": ["COMPLETED", "FAILED", "PAUSED"]
  }
}
```

← Your AWS access key
← Your AWS secret key
← AWS region
← Leave empty for AWS
← Leave empty for AWS
← Bryck path for uploads
← S3 bucket URI
← Bryck path for downloads
← SNS topic ARN (optional)
← SQS queue ARN (optional)
← Email subscribers (optional)
← States to notify on

Security notes: - cloud_ops.json is gitignored — it stays local and won't be committed to version control - Never share this file (contains AWS credentials) - Use AWS IAM best practices: create dedicated user with minimal S3 permissions - For GCP: set cloud_type: "gcp" and provide keyfile path to service account JSON - For Azure: set cloud_type: "azure" and provide tenant_id

Getting AWS credentials: 1. Log in to AWS Console → IAM 2. Create new user with programmatic access 3. Attach policy: **AmazonS3FullAccess** (or more restrictive custom policy) 4. Copy Access Key ID and Secret Access Key 5. Paste into cloud_ops.json

Step 6: Run Your First Format and Mount Now you're ready to format and mount your Bryck.

Typical first-time workflow:

```
# Step 1: Scan (discover attached storage)
python3 bryck_scan.py
```

Step 2: Format (create filesystem)

```
python3 bryck_format.py
```

Step 3: Mount (make it accessible)

```
python3 bryck_mount.py
```

Example session:

```
$ python3 bryck_scan.py
```

```
Connecting to Bryck at 192.168.6.35...
```

```
Authentication successful
```

```
Scan initiated...
```

```
Scan completed (drives detected)
```

```
$ python3 bryck_format.py
```

```
Connecting to Bryck at 192.168.6.35...
```

```
Authentication successful
```

```
Current bryck state: 'Ejected' - ready to format
```

```
Formatting Bryck (RAID 5, ZFS, 256 KB blocks)...
```

```
Configuring store: [ ] 100% (120s/600s)
```

```
Configuration complete
```

```
Formatting filesystem...
```

```
Formatting: [ ] 100% (180s/300s)
```

```
Bryck formatted successfully
```

```
Capacity: 4.0 TB
```

```
RAID: 5
```

```
Filesystem: ZFS
```

```
$ python3 bryck_mount.py
```

```
Connecting to Bryck at 192.168.6.35...
```

```
Authentication successful
```

```
Current bryck state: 'Ejected' - ready to mount
```

```
Mounting Bryck at /bryck...
```

```
Mounting: [ ] 100% (30s/120s)
```

```
Bryck mounted successfully at /bryck
```

```
Mount will persist after reboot (mountonreboot=true)
```

```
$ df -h /bryck
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/bryck	4.0T	0	4.0T	0%	/bryck

Step 7: Verify Everything Works

```
# Check current state
python3 bryck_info.py
```

```
# You should see:
# "State": " Mounted"
```

0.2 Common First-Time Mistakes

Mistake	Impact	Fix
Using hostname instead of IPv4	Connection fails	Use IPv4: 192.168.6.35
Wrong password	401 Unauthorized	Verify with Bryck admin
SSH credentials missing	format/mount/erase fail	Add SSH creds to login.json
Format when “Removed”	Exit 2 error	Run scan first
Mount when “Removed”	Exit 2 error	Run scan then format first
Scan when not “Removed”	Exit 2 error	Drives detected; skip scan
Eject during “Mounting”	Temporary failure	Wait and retry
Eject when “Ejected”	Exit 2 error	Already ejected; continue
Invalid login.json syntax	JSON parse error	Validate with json.tool

0.3 Verify Your login.json is Valid

```
# Validate JSON syntax
python3 -m json.tool login.json

# Should show:
# {
#   "bryckapi_host": "192.168.6.35",
#   ...
# }
# (with no error messages)
```

0.1 Quick Reference Card — Most Common Operations

Copy-paste these commands for common tasks:

First-Time Setup

```
python3 bryck_scan.py      # Step 1: Detect drives (state must be "Removed")
python3 bryck_format.py    # Step 2: Format (state must be "Ejected")
python3 bryck_mount.py     # Step 3: Mount (state must be "Ejected")
```

Check Bryck Status

```
python3 bryck_info.py      # Show state, capacity, health
python3 bryck_network_info.py # Show network configuration
```

Safe Shutdown

```
python3 bryck_eject_unmount.py # Step 1: Unmount (state must be "Mounted")
python3 bryck_remove.py        # Step 2: Remove from system (state must be "Ejected")
```

Wipe and Reformat

```
python3 bryck_eject_unmount.py # Step 1: Unmount (state must be "Mounted")
python3 bryck_erase.py         # Step 2: Wipe data (state must be "Ejected")
python3 bryck_scan.py          # Step 3: Rediscover (state must be "Removed")
python3 bryck_format.py        # Step 4: Reformat (state must be "Ejected")
python3 bryck_mount.py         # Step 5: Remount (state must be "Ejected")
```

System Configuration

```
python3 change_ip.py        # Change management IP (edit change_ip_params.json first)
python3 change_time.py      # Set system time (edit change_time_params.json first)
```

Setup Notifications

```
python3 bryck_cloud_notification_setup.py          # Configure
python3 bryck_cloud_notification_subscribe.py --email user@example.com # Add emails
python3 bryck_cloud_notification_enable.py          # Turn on
```

Cloud Transfer (AWS Example)

```
# Cloud configuration
python3 bryck_cloud_configure.py

# Upload to cloud
python3 bryck_cloud_transfer_initiate.py --mode upload

# Check progress
python3 bryck_cloud_transfer_status.py
```

```
# Download from cloud  
python3 bryck_cloud_transfer_initiate.py --mode download
```

```
# Cloud configuration removal  
python3 bryck_cloud_deconfigure.py --cloud-type aws
```

Get Help

```
python3 bryck_info.py --help # Show all runners and basic help
```

1. Runner Reference — Organized by Category

All runners:

1. Load `login.json` for credentials.
2. Operate the Bryck REST API (plus SSH for some).
3. Return structured output (JSON or formatted text).
4. Exit with a status code:
 - 0 = success
 - 1 = API error
 - 2 = parameter error
 - 3 = validation timeout

1.1 Lifecycle Runners — Format, Mount, Erase

These runners manage the Bryck’s storage lifecycle.

Runner	Purpose	SSH	Precondition	After
<code>bryck_scan.py</code>	Discover drives	Yes	Removed (skips otherwise)	Removed
<code>bryck_format.py</code>	Configure filesystem	Yes	Ejected only	Ejected
<code>bryck_mount.py</code>	Mount at <code>/bryck</code>	Yes	Ejected only	Mounted
<code>bryck_eject_unmount.py</code>	Unmount & eject	No	Mounted only	Ejected
<code>bryck_erase.py</code>	Secure erase	No	Ejected only	Ejected
<code>bryck_remove.py</code>	Unregister	No	Ejected only	Removed

State Precondition Rules (STRICT ENFORCEMENT):

- **`bryck_scan.py`**: Only scans if state is “Removed”
 - Detects drives and transitions from “Removed” → still “Removed”
 - Skips gracefully with success (exit 0) if drives already detected (state is not “Removed”)
- **`bryck_format.py`**: State MUST be “Ejected” (only)
 - Configures filesystem and transitions “Ejected” → still “Ejected”
 - Error if state is “Removed” (must run scan first)
 - Error if state is “Mounted” (must eject first)
- **`bryck_mount.py`**: State MUST be “Ejected” (only)
 - Mounts filesystem and transitions “Ejected” → “Mounted”
 - Error if state is “Removed” (must run scan + format first)
 - Error if state is “Mounted” (already mounted)
- **`bryck_eject_unmount.py`**: State MUST be “Mounted” (only)
 - Ejects and transitions “Mounted” → “Ejected”
 - Special check: Error if state is “Mounting” (cannot eject while mounting)

Example:

```
python3 bryck_info.py
# Output: {"State": " Mounted", "Capacity": "4.0 TB", "Health": "OK"}

python3 bryck_network_info.py
# Output: {"p1": {"ip": "192.168.6.35", "netmask": "255.255.255.0"}}
```

1.3 System Configuration Runners

Configure network and time on the Bryck.

Runner	Purpose	Config File	Notes
change_ip.py	Set IP & network	change_ip_params.json	Configures network interface
change_time.py	Set date/time	change_time_params.json	Manual or NTP sync

Example:

```
# Edit change_ip_params.json, then:
python3 change_ip.py
# Network configured (may need to reconnect)

# Edit change_time_params.json, then:
python3 change_time.py
# Time set to 2026-08-08 14:48:32
```

1.4 Cloud Provider Configuration Runners

Setup, list, and remove cloud providers (AWS/GCP/Azure).

Runner	Purpose	Input	Result
bryck_cloud_configure.py	Apply or update cloud provider credentials	cloud_ops.json	Configured
bryck_cloud_show.py	List all configured cloud providers	—	Display
bryck_cloud_deconfigure.py	Remove cloud provider configuration	-cloud-type	Removed

Example workflow:

```
# Step 1: Create cloud_ops.json with AWS credentials
# (See cloud_ops.json section below)

# Step 2: Configure
python3 bryck_cloud_configure.py
# AWS configured

# Step 3: Verify
python3 bryck_cloud_show.py
# Lists "aws" provider with bucket/region

# Step 4: Remove if needed
python3 bryck_cloud_deconfigure.py --cloud-type aws
# AWS removed
```

1.5 Cloud Transfer Runners — Upload/Download Data

Manage cloud transfers (upload to AWS/GCP/Azure, download, etc.).

Runner	Purpose	Terminal state
bryck_cloud_transfer_start_cli.py	Start a transfer to/from AWS/GCP/Azure	IN_PROGRESS
bryck_cloud_transfer_get_status.py	Get details of progress and stats	—
bryck_cloud_transfer_pause.py	Pause a transfer (resumable)	PAUSED
bryck_cloud_transfer_resume.py	Resume a paused transfer from checkpoint	IN_PROGRESS
bryck_cloud_transfer_cancel.py	Cancel a transfer (cannot resume)	CANCELLED
bryck_cloud_transfer_report.py	Download transfer report ZIP file	—

Valid transfer states:

- IN_PROGRESS — transfer running
- PAUSED — paused (resumable)
- COMPLETED — finished successfully
- FAILED — failed (check report)
- STOPPED — manually stopped

- CANCELLED — cancelled by user

Example workflow:

```
# 1. Configure cloud provider first (see §1.4)
python3 bryck_cloud_configure.py

# 2. Start transfer (reads cloud_ops.json)
python3 bryck_cloud_transfer_initiate.py --mode upload
# Output: Transfer ID 69, polling...

# 3. Check progress anytime
python3 bryck_cloud_transfer_status.py --transfer-id 69
# Output: TRANSFER_ID=69 : IN_PROGRESS : 250GB/1TB (25%)

# 4. If needed, pause/resume
python3 bryck_cloud_transfer_pause.py --transfer-id 69
python3 bryck_cloud_transfer_resume.py --transfer-id 69

# 5. When done, get report
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 69 --report-path .
```

1.6 Notification Runners — Email and SNS Alerts

Setup email notifications for cloud transfer completion/failure.

Runner	Purpose	Input
notification_setup	Configure SNS + emails	cloud_ops.json
notification_subscribe	Add email	–email
notification_subscriber_show	List subscribers	(none)
notification_unsubscribe	Remove email	–email
notification_list	Show config	(none)
notification_enable	Turn on	(none)
notification_disable	Turn off	(none)
notification_delete	Delete config	(none)

Note: All runners prefixed with `bryck_cloud_` and suffixed `.py`

Example workflow:

```
# Step 1: Setup (requires AWS SNS topic ARN)
# Edit cloud_ops.json:
{
```

```

    "notification": {
        "sns_topic": "arn:aws:sns:us-west-1:123456789012:bryck-alerts",
        "emails": ["admin@company.com"]
    }
}

# Step 2: Configure
python3 bryck_cloud_notification_setup.py
# SNS configured

# Step 3: Add more subscribers
python3 bryck_cloud_notification_subscribe.py --email ops@company.com
# ops@company.com subscribed

# Step 4: Verify
python3 bryck_cloud_notification_subscriber_show.py
# Lists: admin@company.com, ops@company.com

# Step 5: Enable
python3 bryck_cloud_notification_enable.py
# Notifications enabled

# Now: When transfers finish, subscribers get notified!

```

2. Configuration files

2.1 login.json

```

{
    "bryckapi_host": "192.168.6.32",
    "bryckapi_scheme": "http",
    "bryckapi_port": "80",
    "bryckapi_username": "admin",
    "bryckapi_password": "<password>",
    "timeout": 300,
    "bryckserver_username": "bryck",
    "bryckserver_password": "<password>"
}

```

All runners load this file (defaults to `./login.json`, override with `--login PATH`).

Field rules

- `bryckapi_host` must be an IPv4 address. Hostnames such as `localhost` are rejected (`ValueError` at load time). Rationale: the runners must work identically whether executed

on the Bryck itself or from any other machine, and paramiko/SSH resolution behaves better with an explicit IP.

- `bryckapi_scheme` accepts `"http"` or `"https"`. TLS certificates are **not** verified by default (`verify=False`); self-signed certs work out of the box.
- `bryckapi_port` / `bryckapi_scheme` are **auto-corrected** when they disagree with the well-known ports: `https + 80` is silently promoted to 443, and `http + 443` demoted to 80. Non-standard combinations (e.g. `https + 8443`) pass through unchanged. An INFO log line notes every correction.
- `bryckserver_username` / `bryckserver_password` are the SSH credentials used by `ssh_runner.py` to reach the Bryck's OS shell (port 22). They are consumed by `bryck_scan.py`, `bryck_format.py`, `bryck_mount.py`, and `bryck_erase.py`. Runners that only touch the REST API (`bryck_info.py`, `change_ip.py`, `change_time.py`, etc.) ignore them.
- `timeout` (integer seconds) is applied to every REST request. Older copies of this document referenced `bryckapi_timeout`; the actual key is `timeout`.

2.2 format_mount_params.json

Without encryption (default):

```
{
  "format": {
    "key_file": "",           // Empty => non-encrypted format. Provide a local path to
    "encryption_option": "",  // Encryption mechanism ("Manual", "KMS"). Empty picks the
    "IoSize": "256",
    "DataSync": "application sync",
    "raid_level": 5,
    "filesystem": "zfs",
    "num_vols": "None"
  },
  "mount": {
    "key_file": "",           // Must match the value used at format time (empty for non-encrypted)
    "encryption_option": "",  // Must match the format value (leave empty for the default)
    "mountonreboot": true,
    "force_check": false
  }
}
```

With encryption (provide key file path):

```
{
  "format": {
    "key_file": "/path/to/bryck.key",
    "encryption_option": "",
    "IoSize": "256",
    "DataSync": "application sync",
    "raid_level": 5,
```

```

        "filesystem": "zfs",
        "num_vols": "None"
    },
    "mount": {
        "key_file": "/path/to/bryck.key",
        "encryption_option": "",
        "mountonreboot": true,
        "force_check": false
    }
}

```

Note: `key_file` — empty for non-encrypted formats, or a local file path to encrypt the Bryck with that key. `encryption_option` — selects the encryption mechanism ("Manual", "KMS"); when left empty the default "Manual" is applied. The mount `key_file` and `encryption_option` must match the values used during format.

Only used by `bryck_format.py` and `bryck_mount.py`. Override the path with `--params PATH`.

key_file semantics (both format and mount):

Value	Behaviour
<code>"", null, "None", "null"</code>	No key sent; operation proceeds without encryption.
Local path that does not exist	Warning logged, operation proceeds without encryption.
Local path that exists	File is uploaded via SFTP (paramiko) using <code>bryckserver_*</code> credentials to <code>/opt/bryck/bryckapi/downloads/keyfile</code> , then made world-readable via a remote <code>sudo -n chmod 0644</code> . That server-side path is passed to the API. Passwordless <code>sudo</code> for <code>chmod</code> is required on the Bryck for <code>bryckserver_username</code> .

2.3 change_ip_params.json

```

{
    "interface_name": "p1",
    "dhcp": false,
    "ip": "11.11.11.32",
    "netmask": "255.255.255.0",
    "gateway": null,
    "nameservers": ["8.8.8.8"],
    "ntp_server": "pool.ntp.org",
    "mtu": 1500
}

```

Used by `change_ip.py`. The runner auto-picks the logical-card UUID from `result.logical_cards` (first key). All fields are forwarded to `POST /api/network/configure`; only `interface_name`, `ip`,

and `netmask` are cross-checked during validation against `result.server_info.ethernet[]`.

2.4 change_time_params.json

```
{
  "option": "Manual",
  "date": "07/17/2026",
  "time": "14:48:32",
  "ntp_server": "None"
}
```

Used by `change_time.py`.

- `option = "Manual"` — `date` (MM/DD/YYYY) and `time` (HH:MM:SS, 24h) from the JSON are sent verbatim. `ntp_server` is forwarded but is typically ignored server-side in Manual mode.
- `option = "NTP"` — payload is always `{option:"NTP", date:null, time:null, ntp_server:"time.google.com"}`. The JSON `date`, `time`, and `ntp_server` values are ignored.

2.5 cloud_ops.json

First-time setup:

```
# Copy the example template
cp cloud_ops.example.json cloud_ops.json
# Edit with your real credentials
$EDITOR cloud_ops.json
```

Security note: `cloud_ops.json` is gitignored and should contain real credentials. Never commit it to version control. The repository includes `cloud_ops.example.json` with dummy values for reference.

Example structure:

```
{
  "cloud_type": "aws",
  "access_key_id": "AKIAIOSFODNN7EXAMPLE",
  "secret_access_key": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY",
  "region": "us-east-1",
  "keyfile": "",
  "tenant_id": "",
  "bryck_src": "/bryck/source",
  "cloud_bucket": "s3://my-bucket/path",
  "bryck_dst": "/bryck/dest",
  "notification": {
    "sns_topic": "arn:aws:sns:us-east-1:123456789012:example-topic",
  }
}
```

```

    "sqs_queue": "arn:aws:sqs:us-east-1:123456789012:example-queue",
    "emails": ["user@example.com"],
    "states": ["COMPLETED", "FAILED", "PAUSED"]
}
}

```

Only used by `bryck_cloud_transfer.py`. Override the path with `--params PATH`.

Field reference

Field	Purpose
<code>cloud_type</code>	Cloud provider — one of <code>aws</code> , <code>gcp</code> , <code>azure</code> (case-insensitive).
<code>access_key_id</code>	Access key / account name (sent as <code>username</code>). Required for AWS and Azure.
<code>secret_access_key</code>	Secret key (sent as <code>keyid</code>). Required for AWS and Azure.
<code>region</code>	Cloud region. AWS-only; required (e.g. <code>us-east-1</code> , <code>us-west-1</code>).
<code>keyfile</code>	Local path to the service-account JSON. Required for GCP; ignored for AWS / Azure.
<code>tenant_id</code>	Azure tenant ID. Required for Azure; ignored for AWS / GCP.
<code>bryck_src</code>	Source path on the Bryck for the upload leg (e.g. <code>/bryck/source</code>). Required.
<code>cloud_bucket</code>	Cloud URI used as the upload destination and the download source (e.g. <code>s3://bucket/path</code>). Required.
<code>bryck_dst</code>	Destination path on the Bryck for the download leg (e.g. <code>/bryck/dest</code>). Required.

Per-cloud requirements

<code>cloud_type</code>	Required	Optional	Default
<code>aws</code>	<code>access_key_id</code> , <code>secret_access_key</code> , <code>region</code>	—	—
<code>gcp</code>	<code>keyfile</code> (path on the machine running the runner)	—	—
<code>azure</code>	<code>access_key_id</code> , <code>secret_access_key</code> , <code>tenant_id</code>	—	—

Missing required fields (or an unsupported `cloud_type`) cause the runner to exit 2 before making any REST call.

GCP keyfile transfer — `keyfile` is a path on the machine executing `bryck_cloud_transfer.py`. The runner SFTP's it to `/opt/bryck/bryckapi/downloads/deployment/.gcloud/<basename>` on the Bryck, then only the `basename` is sent to the REST API. Because `.gcloud/` is root-owned on stock installs, the runner uses `sudo -n mkdir -p`, `sudo -n mv`, and `sudo -n chmod 0644`. Configure passwordless `sudo` for `bryckserver_username` on the Bryck for those three commands (same visudo pattern as §7).

Notification section (optional) — `bryck_cloud_notification_setup.py` (§5.17) loads the optional notification object:

```

{
  "notification": {
    "sns_topic": "arn:aws:sns:us-east-1:123456789012:MyTopic",

```

```

    "sqs_queue": "arn:aws:sqs:us-east-1:123456789012:MyQueue",
    "emails": ["user1@example.com", "user2@example.com"],
    "states": ["COMPLETED", "FAILED", "PAUSED"]
  }
}

```

Field	Purpose
<code>sns_topic</code>	AWS SNS topic ARN (optional if <code>sqs_queue</code> provided).
<code>sqs_queue</code>	AWS SQS queue ARN (optional if <code>sns_topic</code> provided).
<code>emails</code>	List of email addresses to notify (optional ; leave empty when using SNS/SQS only).
<code>states</code>	List of transfer states to notify on (optional; default: all states).

At least ONE of `sns_topic` or `sqs_queue` must be provided. Valid states are **COMPLETED**, **FAILED**, **PAUSED**.

3. Bryck state machine

The runners read `result.bryck_info.State` from `/api/config/info`. The API returns state strings **with a leading space** — the runners compare against exactly those strings:

- " Mounted"
- " Ejected"
- " Removed"

Required state per runner

Runner	Precondition	On mismatch
scan	any (runs only when " Removed")	—
format	not " Mounted"	exit 2
mount	not " Mounted"	exit 2
eject_unmount	" Mounted"	exit 2
erase	" Ejected"	exit 2
remove	" Ejected"	exit 2
change_ip	any	—
change_time	any	—
info	any	—
network_info	any	—
report	any	—

```
cloud_transfer_initiate | " Mounted" | exit 2 |
cloud_transfer_pause | any | — |
cloud_transfer_resume | any | — |
cloud_transfer_cancel | any | — |
cloud_configure | any | — |
cloud_show | any | — |
cloud_deconfigure | any | — |
cloud_transfer_status | any | — |
cloud_transfer_report | any | — |
```

Note: All runner names are prefixed with `bryck_` and suffixed with `.py` (e.g., `scan = bryck_scan.py`)

`bryck_scan.py`, `bryck_format.py`, `bryck_mount.py`, and `bryck_erase.py` all wrap their call to `/api/config/scan` in the same guard: the scan is issued **only** when `bryck_info.State == " Removed"`. In any other state the drives are already detected, so the runner logs `Bryck drives are detected (state=<...>); skipping scan` and moves on to the next step (or, for `bryck_scan.py`, exits 0).

`bryck_eject_unmount.py` accepts **either** `" Ejected"` or `" Removed"` as a successful eject terminal state — a Bryck that gets physically pulled from the tray during the eject window transitions straight to `" Removed"`, and the runner treats that as an eject that still landed.

4. Typical lifecycles

4.1 First-time provisioning

```
bryck_scan.py      # optional sanity check
bryck_format.py    # requires state != " Mounted"
bryck_mount.py     # requires state != " Mounted"
```

4.2 Safe unmount for shipping

```
bryck_eject_unmount.py  # " Mounted" -> " Ejected" (or " Removed")
bryck_remove.py         # " Ejected" -> " Removed" (or UUID gone)
```

4.3 Wipe and reformat

```
bryck_eject_unmount.py  # " Mounted" -> " Ejected" (or " Removed")
bryck_erase.py          # " Ejected" -> data cleared
bryck_scan.py           # rediscover (scan only if state is " Removed")
bryck_format.py         # reformat (auto-skips scan when drives are detected)
bryck_mount.py          # remount (auto-skips scan when drives are detected)
```

4.4 System configuration (state-independent)

```
change_ip.py           # /api/network/configure (edit change_ip_params.json first)
change_time.py         # /api/settings/set_date (edit change_time_params.json first)
```

Both runners work in any Bryck state. `change_ip.py` polls `server_info.ethernet[]` until the target interface reflects the requested ip and netmask. `change_time.py` polls `server_info.server_time` for Manual mode; NTP mode skips validation.

4.5 Inspection / diagnostics (read-only, state-independent)

```
bryck_info.py          # full result of /api/config/info
bryck_network_info.py  # only result.server_info.ethernet
bryck_report.py --output-dir <dir> # generate + download bryck_report.tgz
```

All three are read-only from the Bryck's perspective (`bryck_report.py` triggers a `CAPTURE_BRYCK_STATE` task but leaves stores untouched).

5. CLI

All runners share the same interface:

```
python3 <runner>.py [--login PATH] [--params PATH]
```

- `--login` defaults to `./login.json`.
- `--params` is accepted by `bryck_format.py`, `bryck_mount.py`, `change_ip.py`, and `change_time.py` (each defaults to its own per-runner JSON).

5.1 bryck_scan.py

```
# Default (login.json alongside the script)
```

```
python3 bryck_scan.py
```

```
# Custom credentials
```

```
python3 bryck_scan.py --login /etc/bryck/prod-login.json
```

Calls `/api/config/scan` only when `bryck_info.State == "Removed"`. In any other state the drives are already detected, so the runner displays `Drives already detected (Bryck state: '<state>')`. No need to scan. and exits 0 without hitting the API.

5.2 bryck_format.py

```
# Default (login.json + format_mount_params.json)
```

```
python3 bryck_format.py
```

```
# Alternate params file (e.g. RAID-6 profile)
```

```
python3 bryck_format.py --params ./raid6-params.json
```

```
# Both custom
```

```
python3 bryck_format.py --login /etc/bryck/prod-login.json \  
                        --params ./raid6-params.json
```

Refuses to run if `bryck_info.State == "Mounted"` (exit 2).

5.3 bryck_mount.py

```
python3 bryck_mount.py
```

```
python3 bryck_mount.py --params ./mount-noreboot.json
```

The mount point is hardcoded to `/bryck`; edit `format_mount_params.json` → mount block to change `key_file`, `mountonreboot`, `force_check`, or `encryption_option`.

5.4 bryck_eject_unmount.py

```
python3 bryck_eject_unmount.py
python3 bryck_eject_unmount.py --login /etc/bryck/prod-login.json
```

Requires `bryck_info.State == " Mounted"`; exits 2 otherwise. The runner treats **either** " Ejected" or " Removed" (with the leading space) as a successful eject terminal state, so a Bryck that gets physically pulled during the eject window still returns 0.

5.5 bryck_erase.py

```
python3 bryck_erase.py
```

Requires `bryck_info.State == " Ejected"`; exits 2 otherwise. Destructive — wipes the store.

5.6 bryck_remove.py

```
python3 bryck_remove.py
```

Requires `bryck_info.State == " Ejected"`; exits 2 otherwise.

5.7 change_ip.py

```
# Default (change_ip_params.json alongside the script)
python3 change_ip.py
```

```
# Alternate params file
python3 change_ip.py --params ./p1-static.json
```

Example `change_ip_params.json` for a static interface:

```
{
  "interface_name": "p1",
  "dhcp": false,
  "ip": "11.11.11.32",
  "netmask": "255.255.255.0",
  "gateway": null,
  "nameservers": ["8.8.8.8"],
  "ntp_server": "pool.ntp.org",
  "mtu": 1500
}
```

Example for DHCP:

```
{
  "interface_name": "oob_net0",
  "dhcp": true,
  "ip": null,
  "netmask": null,
  "gateway": null,
  "nameservers": null,
  "ntp_server": null,
  "mtu": 1500
}
```

Warning: configuring the management interface used for the current session will drop connectivity mid-run and validation will time out (exit 3). The change still lands — verify manually on the new IP.

5.8 change_time.py

Default (change_time_params.json alongside the script)

```
python3 change_time.py
```

Alternate params file

```
python3 change_time.py --params ./ntp-mode.json
```

Example Manual mode:

```
{
  "option": "Manual",
  "date": "07/17/2026",
  "time": "14:48:32",
  "ntp_server": "None"
}
```

Example NTP mode (all JSON fields except `option` are ignored; `ntp_server` is always sent as `"time.google.com"`):

```
{
  "option": "NTP",
  "date": null,
  "time": null,
  "ntp_server": "None"
}
```

After a successful `set_date` in Manual mode the runner re-logs in automatically to refresh its JWT (the old token is invalidated by the clock jump — see §8).

5.9 bryck_info.py

```
# Print the full `result` of /api/config/info as pretty JSON  
python3 bryck_info.py
```

```
# Also write the same JSON to a file  
python3 bryck_info.py --output /tmp/bryck_info.json
```

```
# Custom credentials  
python3 bryck_info.py --login /etc/bryck/prod-login.json
```

Useful for debugging — everything the state-machine and validators read (bryck_info.State, logical_cards, server_info, tray_info, ...) comes from this response.

5.10 bryck_network_info.py

```
# Print result.server_info.ethernet  
python3 bryck_network_info.py
```

```
# Save to a file as well  
python3 bryck_network_info.py --output /tmp/ethernet.json
```

Same source as bryck_info.py, filtered down to the network-interface array (ip, netmask, gateway, mac, mtu, interface_name, ...). Useful for sanity-checking a change_ip.py run.

5.11 bryck_report.py

```
# Generate and download bryck_report.tgz into /tmp/reports/  
python3 bryck_report.py --output-dir /tmp/reports
```

```
# Custom filename  
python3 bryck_report.py --output-dir /tmp/reports --filename my_report.tgz
```

```
# Custom credentials  
python3 bryck_report.py --output-dir /tmp/reports \  
    --login /etc/bryck/prod-login.json
```

Flow:

1. POST /api/tasks/capture_bryck_state — starts the capture task.
2. Polls GET /api/tasks/list?task_type=CAPTURE_BRYCK_STATE every 1 s (budget: REPORT_TIMEOUT = 600 s) until result[0].state == "COMPLETED".
3. GET /api/download?name=bryck_report — streams the tgz.
4. Saves to <output-dir>/<filename> (default filename: bryck_report.tgz).

The output directory is created if it doesn't exist. Exits 3 if the capture task hasn't reached COMPLETED within 600 s.

5.12 bryck_cloud_transfer_initiate.py

--mode is **mandatory** — every invocation must state whether it is running an upload, a download, or both. There is no default.

Upload only (bryck_src -> cloud_bucket). bryck_dst is not consulted.

```
python3 bryck_cloud_transfer_initiate.py --mode upload
```

Download only (cloud_bucket -> bryck_dst). bryck_src is not consulted.

```
python3 bryck_cloud_transfer_initiate.py --mode download
```

Both directions in one run (previous default behaviour).

```
python3 bryck_cloud_transfer_initiate.py --mode both
```

Alternate params file (e.g. an Azure profile) + custom credentials.

```
python3 bryck_cloud_transfer_initiate.py --mode both \  
  --params ./azure-cloud_ops.json \  
  --login /etc/bryck/prod-login.json
```

AWS cloud_ops.json example:

```
{  
  "cloud_type": "aws",  
  "access_key_id": "AKIA...",  
  "secret_access_key": "wJalr...",  
  "region": "us-west-1",  
  "bryck_src": "/bryck/source",  
  "cloud_bucket": "s3://my-bucket/path",  
  "bryck_dst": "/bryck/dest"  
}
```

GCP cloud_ops.json example (only keyfile is required for auth):

```
{  
  "cloud_type": "gcp",  
  "keyfile": "/home/admin/gcp_sa.json",  
  "bryck_src": "/bryck/source",  
  "cloud_bucket": "gs://my-bucket/path",  
  "bryck_dst": "/bryck/dest"  
}
```

Azure cloud_ops.json example:

```
{  
  "cloud_type": "azure",  
  "access_key_id": "<storage-account-name>",
```

```

"secret_access_key": "<storage-account-key>",
"tenant_id": "<tenant-uuid>",
"bryck_src": "/bryck/source",
"cloud_bucket": "https://<account>.blob.core.windows.net/<container>/path",
"bryck_dst": "/bryck/dest"
}

```

Behaviour:

- **Runs upload, download, or both — selected via mandatory `--mode`.** This runner configures the cloud provider, then kicks off whichever transfer(s) `--mode` requests and confirms each entered `IN_PROGRESS` on the server. It does **not** wait for any transfer to reach `COMPLETED`. Cloud transfer duration scales with payload size (MBs → TBs); pinning a fixed completion budget is not viable. On success the runner prints the transfer_id(s) that were actually started in a highlighted banner along with the exact `bryck_cloud_transfer_status.py` command to check progress — copy those IDs and use §5.15 to poll for the final state. Omitting `--mode` fails immediately with `argparse` exit 2.
- **Per-mode `cloud_ops.json` field requirements** — `cloud_bucket` is always required; the runner's mode-aware validation only demands the source / destination path it will actually use:

Mode	Required	Ignored
upload	bryck_src, cloud_bucket	bryck_dst
download	cloud_bucket, bryck_dst	bryck_src
both	bryck_src, cloud_bucket, bryck_dst	—

A missing required field exits 2 with a message naming the mode and the offending field(s). Ignored fields may be present (they are silently unused) or omitted / left empty.

- **" Mounted" precondition** — the runner reads `bryck_info.State` immediately after login and refuses to start the transfer unless the Bryck is " Mounted" (exit 2). This is the same guard used by `bryck_eject_unmount.py`.
- **Idempotent configuration** — if a cloud config for the same `cloud_type` already exists, `_validate_cloud_configured` returns `True` immediately; no error, no re-post is attempted after that.
- **`IN_PROGRESS` start budget** — `TRANSFER_START_TIMEOUT` seconds (default 120). If the server accepted the request but the transfer never advances into `IN_PROGRESS` within that window, the runner exits 3.
- **Fast fail on terminal states** — if the server reports `FAILED`, `STOPPED`, or `CANCELLED` while the runner is polling for `IN_PROGRESS`, the runner exits 4 immediately without waiting out the timeout.
- **Config left in place** — the runner does **not** call `remove_cloud_config` on exit. The Bryck's transfer engine still needs the credentials to finish the transfers in the background after the

runner returns; tearing them down early would break the in-flight work. Remove the config later via `bryck_cloud_deconfigure.py` (§5.14) when it is no longer needed (e.g. to rotate credentials).

5.13.1 `bryck_cloud_transfer_pause.py`

```
# Pause an active transfer
python3 bryck_cloud_transfer_pause.py --transfer-id 69

# Custom credentials
python3 bryck_cloud_transfer_pause.py --transfer-id 69 \
    --login /etc/bryck/prod-login.json
```

Flow:

1. Log in to the Bryck REST API.
2. POST `/api/bcloud/pause_transfer` with the specified transfer ID.
3. Poll GET `/api/bcloud/status_transfer` until the transfer enters PAUSED state (budget: `VALIDATION_TIMEOUT = 60 s`).

Behaviour:

- `--transfer-id` is **required**. The runner accepts any string value and passes it to the API; validation happens server-side.
- No state precondition — works on any transfer that the API accepts for pausing (typically `IN_PROGRESS` transfers).
- Exits 1 if the API returns non-200 status.
- Exits 3 if the transfer does not reach `PAUSED` state within 60s.
- Progress bar displays “Waiting for transfer to pause” during validation.

5.13.2 `bryck_cloud_transfer_resume.py`

```
# Resume a paused transfer
python3 bryck_cloud_transfer_resume.py --transfer-id 69

# Custom credentials
python3 bryck_cloud_transfer_resume.py --transfer-id 69 \
    --login /etc/bryck/prod-login.json
```

Flow:

1. Log in to the Bryck REST API.
2. POST `/api/bcloud/resume_transfer` with the specified transfer ID.
3. Poll GET `/api/bcloud/status_transfer` until the transfer enters `IN_PROGRESS` or `COMPLETED` state (budget: `VALIDATION_TIMEOUT = 60 s`).

Behaviour:

- `--transfer-id` is **required**. The runner accepts any string value and passes it to the API; validation happens server-side.
- No state precondition — works on any transfer that the API accepts for resuming (typically PAUSED transfers).
- Accepts both IN_PROGRESS and COMPLETED as success states (transfer may complete very quickly after resuming).
- Exits 1 if the API returns non-200 status.
- Exits 3 if the transfer does not reach IN_PROGRESS/COMPLETED state within 60s.
- Progress bar displays “Waiting for transfer to resume” during validation.

5.13.3 bryck_cloud_transfer_cancel.py

```
# Cancel a transfer
python3 bryck_cloud_transfer_cancel.py --transfer-id 69

# Custom credentials
python3 bryck_cloud_transfer_cancel.py --transfer-id 69 \
    --login /etc/bryck/prod-login.json
```

Flow:

1. Log in to the Bryck REST API.
2. POST `/api/bcloud/cancel_transfer` with the specified transfer ID.
3. Poll GET `/api/bcloud/status_transfer` until the transfer enters CANCELLED state (budget: VALIDATION_TIMEOUT = 60 s).

Behaviour:

- `--transfer-id` is **required**. The runner accepts any string value and passes it to the API; validation happens server-side.
- No state precondition — works on any transfer that the API accepts for cancellation.
- Cancellation is **irreversible** — a cancelled transfer cannot be resumed.
- Exits 1 if the API returns non-200 status.
- Exits 3 if the transfer does not reach CANCELLED state within 60s.
- Progress bar displays “Waiting for transfer to cancel” during validation.

5.13.4 bryck_cloud_configure.py

```
# Configure cloud provider from cloud_ops.json
python3 bryck_cloud_configure.py

# Custom params file (e.g. Azure profile) + custom credentials
python3 bryck_cloud_configure.py \
    --params ./azure-cloud_ops.json \
    --login /etc/bryck/prod-login.json
```

Flow:

1. Load cloud configuration parameters from `cloud_ops.json` (or `--params` path).
2. Validate required fields for the specified cloud type (AWS/GCP/Azure).
3. For GCP: Upload the service-account keyfile to the Bryck via SFTP into `/opt/bryck/bryckapi/downloads/`.
4. Log in to the REST API.
5. `POST /api/bcloud/config` to configure the cloud provider.
6. Poll `GET /api/bcloud/config_list` until the provider appears in `result` (budget: `CONFIGURE_TIMEOUT = 60 s`).
7. Print success message and exit.

Behaviour:

- **Cloud-type specific validation** — the runner checks `cloud_type` field in `cloud_ops.json` and enforces required fields:
 - **AWS**: `access_key_id` + `secret_access_key` + `region` all required.
 - **GCP**: `keyfile` required (local path to service-account JSON). The file is validated locally before upload.
 - **Azure**: `access_key_id` + `secret_access_key` + `tenant_id` required.
- **No transfer fields needed** — unlike `bryck_cloud_transfer_initiate.py`, this runner does **not** require `bryck_src`, `cloud_bucket`, or `bryck_dst` fields. It only configures the cloud provider.
- **No state precondition** — cloud configuration is safe in any Bryck lifecycle state.
- **Config left in place** — the runner does **not** remove the configuration on exit. Use `bryck_cloud_deconfigure.py` (§5.14) to remove it later.
- **GCP keyfile placement** — for GCP, the local keyfile is uploaded to the Bryck via SFTP. The file is staged in `/tmp`, then moved to `/opt/bryck/bryckapi/downloads/deployment/.gcloud/` with `chmod 0644` using `sudo` commands.
- **Idempotent** — if a cloud config for the same `cloud_type` already exists, the validation passes immediately.
- Exits 2 if `cloud_ops.json` is missing/invalid or required fields are missing.
- Exits 3 if GCP keyfile upload fails, API returns non-200 status, or validation times out after 60s.
- Formatted error display with unicode box and emoji for all error conditions.
- Progress bar displays “Waiting for cloud configuration” during validation.

5.13.5 `bryck_cloud_show.py`

Display all configured cloud providers

```
python3 bryck_cloud_show.py
```

Custom credentials

```
python3 bryck_cloud_show.py --login /etc/bryck/prod-login.json
```

Save output to file

```
python3 bryck_cloud_show.py --output cloud_configs.json
```

Flow:

1. Log in to the Bryck REST API.
2. GET `/api/bcloud/config_list` to retrieve all configured cloud providers.
3. Display each configuration in a formatted multi-line layout with dividers and aligned fields (similar to `bryck_cloud_transfer_status.py`).
4. Optionally write the raw JSON to a file if `--output` is specified.

Behaviour:

- **Read-only operation** — this runner does not modify any cloud configurations. It only displays what is currently configured.
- **No state precondition** — works in any Bryck lifecycle state.
- **Display format** — outputs each cloud configuration in a multi-line format with 80-character dividers () and aligned field labels. Sorted by configuration ID for consistent output.
- **Empty list handling** — if no cloud providers are configured, prints “No cloud providers configured.” and exits 0.
- **Optional JSON output** — use `--output PATH` to write the raw JSON to a file (formatted output is always shown to stdout). Useful for scripting or logging.
- Each configuration displays:
 - `CLOUD_TYPE`: Cloud provider type (AWS, GCP, AZURE)
 - `CONFIG_ID`: Configuration ID
 - `USERNAME`: Access key ID or account name
 - `REGION`: AWS region (displayed only for AWS configurations)
 - `CONFIGURED_AT`: Timestamp formatted as YYYY-MM-DD HH:MM:SS
- **Security note** — the API does **not** return secrets (`secret_access_key`, keyfile contents, etc.) in the response.
- Exits 0 on success (including when no providers are configured).
- Exits 1 if API returns non-200 status or response parsing fails.
- Exits 2 if `login.json` is invalid.
- Formatted error display with unicode box and emoji for all error conditions.

5.14 `bryck_cloud_deconfigure.py`

```
# Remove an AWS cloud configuration
python3 bryck_cloud_deconfigure.py --cloud-type aws

# GCP / Azure work the same way
python3 bryck_cloud_deconfigure.py --cloud-type gcp
python3 bryck_cloud_deconfigure.py --cloud-type azure

# Custom credentials
python3 bryck_cloud_deconfigure.py --cloud-type aws \
    --login /etc/bryck/prod-login.json
```

Flow:

1. Log in to the Bryck REST API.
2. POST `/api/bcloud/config_remove` for the requested `--cloud-type`.
3. Poll GET `/api/bcloud/config_list` until the cloud type no longer appears in `result` (budget: `DECONFIGURE_TIMEOUT = 60 s`).

Behaviour:

- `--cloud-type` is **required** and restricted to `aws / gcp / azure` (case-insensitive; normalised to lowercase before the REST call). Any other value is rejected by argparse before any network I/O happens.
- No SSH / GCP keyfile handling — removal is a REST-only op. Any keyfile previously staged at `/opt/bryck/bryckapi/downloads/deployment/.gcloud/` is left on disk; clean it up manually if credentials must be rotated.
- No state precondition — the runner works regardless of `bryck_info.State`.
- Exits 3 if `remove_cloud_config` returns a non-2xx or if the cloud type is still listed in `config_list` after `DECONFIGURE_TIMEOUT`.

5.15 `bryck_cloud_transfer_status.py`

```
# Show all transfers (no filtering).
python3 bryck_cloud_transfer_status.py

# Show one specific transfer.
python3 bryck_cloud_transfer_status.py --transfer-id 69

# Show all transfers with state=COMPLETED (case-insensitive).
python3 bryck_cloud_transfer_status.py --state COMPLETED

# Show all IN_PROGRESS transfers.
python3 bryck_cloud_transfer_status.py --state in_progress

# Custom credentials
python3 bryck_cloud_transfer_status.py --transfer-id 69 \
    --login /etc/bryck/prod-login.json
```

Read-only status runner intended as the follow-up to `bryck_cloud_transfer.py` (§5.13). Copy the transfer IDs printed in that runner's success banner and pass them here to check progress and/or final state.

Flow:

1. Log in to the Bryck REST API.
2. **With** `--transfer-id` — GET `/api/bcloud/status_transfer` for that ID and print a single summary line. The endpoint returns `result` as a list of dicts; the runner accepts both list-shaped and bare-dict responses.

3. **With `--state`** — POST `/api/bcloud/list_transfer` for all transfers, filter entries by the specified state (case-insensitive), and print one summary line per matching transfer. If the supplied state is not in the canonical set (IN_PROGRESS, COMPLETED, PAUSED, FAILED, STOPPED, CANCELLED), the runner logs a warning but still applies the filter (tolerates API evolution and operator typos).
4. **With neither flag** — POST `/api/bcloud/list_transfer` for all transfers and print one summary line per transfer (no filtering).

`--transfer-id` and `--state` are **mutually exclusive**. Providing both fails immediately with argparse exit 2.

Output format — multi-line with 80-character dividers between entries; byte counts converted to GB with 2 decimals; whole percentages render as an integer, others as `.2f`; missing fields render as `- / - GB`:

```
TRANSFER_ID      : 69
STATE             : PAUSED
PROGRESS          : 0.00 GB / 0.00 GB (0% completed)
SOURCE            : /bryck/api-test
DESTINATION       : s3://kunshi-testbucket/api-test49
STARTED_AT        : 2026-07-28 08:37:09.100016+00:00
LAST_UPDATED      : 2026-07-28 08:37:12.462332+00:00
```

Behaviour:

- No state precondition. No SSH. No polling — a single REST call per invocation.
- **Output is sorted by transfer ID numerically** (ascending: 1, 2, 3... 70, 71) for both filtered (`--state`) and unfiltered (no flags) results. Improves readability when tracking multiple transfers.
- Empty result (`--transfer-id` looks up a missing ID, or `--state` / no-filter run has no matching entries) is **not** an error. The runner logs `Transfer <id> not found on the Bryck or No cloud transfers found with state=<state>` or `No cloud transfers found` and exits 0.
- Valid states (canonical set — API may accept others): IN_PROGRESS, COMPLETED, PAUSED, FAILED, STOPPED, CANCELLED.
- Exit codes are simpler than the write-side runners: 0 for success (including “no match” / “not found”); 1 for HTTP / API failure. There is no 3 / 4 (no ticker, no state machine); argparse exit 2 is only triggered by mutual-exclusivity violations.

5.16 bryck_cloud_transfer_report.py

```
# Default destination: ./cloud_transfer_report_<id>.zip in cwd.
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 67
```

```
# Directory: same filename inside that directory.
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 67 \
```

```

--report-path /home/atanu/project

# Full file path (parent dir must already exist).
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 67 \
    --report-path /var/log/bryck/report-67.zip

# Custom credentials
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 67 \
    --report-path /home/atanu/project \
    --login /etc/bryck/prod-login.json

```

Downloads the cloud-transfer report ZIP produced by the Bryck (contains `transfer_summary.txt`, `transfer_report.json`, ...). Extraction and parsing are **not** performed by this runner.

Flow:

1. Log in to the Bryck REST API.
2. GET `/api/download?name=cloud_log&type=<transfer_id>` as a **streamed** response.
3. Write the body to disk in 8 KiB chunks (`resp.iter_content(chunk_size=8192)`).

`--report-path` resolution (`_resolve_report_path`):

Value	Result
omitted	<code>./cloud_transfer_report_<id>.zip</code> in cwd
existing directory	<code><dir>/cloud_transfer_report_<id>.zip</code>
anything else	treated as a full file path (parent dir must exist)

Exit codes: 0 success · 1 API/HTTP failure · 2 bad CLI args or the destination directory does not exist · 3 filesystem write error or a 0-byte response.

Note: downloading a streamed body requires `ApiSession.get(..., stream=True)`. If your `session.py` predates the fix for that keyword, the runner will crash with `TypeError: ApiSession.get() got an unexpected keyword argument 'stream'` — redeploy the current `session.py`.

5.17 bryck_cloud_notification_setup.py

```

# Setup SNS/SQS notification configuration from cloud_ops.json
python3 bryck_cloud_notification_setup.py

# Custom cloud_ops.json
python3 bryck_cloud_notification_setup.py --params /etc/bryck/cloud_ops.json

# Custom credentials
python3 bryck_cloud_notification_setup.py \

```

```
--params cloud_ops.json \
--login /etc/bryck/prod-login.json
```

Configures cloud transfer notifications via AWS SNS and/or SQS. Parameters are loaded from `cloud_ops.json` (not CLI args).

Cloud_ops.json notification section:

```
{
  "notification": {
    "sns_topic": "arn:aws:sns:us-east-1:123456789012:MyTopic",
    "sqs_queue": "arn:aws:sqs:us-east-1:123456789012:MyQueue",
    "emails": ["user1@example.com", "user2@example.com"],
    "states": ["COMPLETED", "FAILED", "PAUSED"]
  }
}
```

Flow:

1. Log in to the Bryck REST API.
2. Load notification section from `cloud_ops.json`.
3. Validate: at least ONE of `sns_topic/sqs_queue` is required; `emails` is optional; `states` array contains only valid states (COMPLETED/FAILED/PAUSED).
4. POST `/api/bcloud/notification_setup` with configuration.
5. Validate via GET `/api/bcloud/notification_list` until config appears (60s timeout).
6. Display configuration in multi-line format.

Behaviour:

- `sns_topic` and `sqs_queue` are optional but at least one must be provided.
- `emails` is optional; when omitted, notifications are delivered via SNS/SQS only.
- `states` is optional; defaults to all states (COMPLETED, FAILED, PAUSED).
- No state precondition — works in any Bryck lifecycle state.
- Config left in place — to remove, use `bryck_cloud_notification_delete.py` (§5.24).
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if `cloud_ops.json` is missing/invalid or required fields missing.
- Exits 3 if validation times out after 60s.
- Formatted error display with unicode box and emoji.
- Progress bar displays “Validating notification setup” during validation.

5.18 bryck_cloud_notification_subscribe.py

Subscribe single email

```
python3 bryck_cloud_notification_subscribe.py --email user@example.com
```

```

# Subscribe multiple emails (repeatable)
python3 bryck_cloud_notification_subscribe.py \
    --email user1@example.com \
    --email user2@example.com

# Subscribe multiple emails (comma-separated)
python3 bryck_cloud_notification_subscribe.py \
    --emails user1@example.com,user2@example.com

# Custom credentials
python3 bryck_cloud_notification_subscribe.py \
    --email user@example.com \
    --login /etc/bryck/prod-login.json

```

Adds email addresses to the notification subscriber list.

Flow:

1. Log in to the Bryck REST API.
2. Collect emails from `--email` (repeatable) and/or `--emails` (comma-separated).
3. Remove duplicates.
4. POST `/api/bcloud/notification_subscribe` with email list.
5. Validate via GET `/api/bcloud/notification_subscribers`.
6. Display subscriber list in multi-line table format.

Behaviour:

- `--email` and `--emails` can be used together; duplicates are automatically removed.
- No state precondition — works in any Bryck lifecycle state.
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if no emails provided or `login.json` is invalid.
- Exits 3 if validation failed.
- Formatted error display with unicode box and emoji.

5.19 bryck_cloud_notification_subscriber_show.py

```

# Display all subscribers
python3 bryck_cloud_notification_subscriber_show.py

# Save JSON to file
python3 bryck_cloud_notification_subscriber_show.py \
    --output subscribers.json

# Custom credentials
python3 bryck_cloud_notification_subscriber_show.py \
    --login /etc/bryck/prod-login.json

```

Displays the list of email subscribers for notifications.

Flow:

1. Log in to the Bryck REST API.
2. GET `/api/bcloud/notification_subscribers`.
3. Display subscriber list in multi-line table format (sorted by subscriber ID).
4. Optionally write raw JSON to file if `--output` is specified.

Behaviour:

- Read-only operation — does not modify subscribers.
- No state precondition — works in any Bryck lifecycle state.
- Display format: table with columns `SUBSCRIBER_ID`, `EMAIL`, `SUBSCRIBED_AT`.
- Empty result prints “No subscribers found” and exits 0.
- Optional JSON output via `--output PATH`.
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if `login.json` is invalid.
- Formatted error display with unicode box and emoji.

5.20 `bryck_cloud_notification_unsubscribe.py`

Unsubscribe an email

```
python3 bryck_cloud_notification_unsubscribe.py --email user@example.com
```

Custom credentials

```
python3 bryck_cloud_notification_unsubscribe.py \  
    --email user@example.com \  
    --login /etc/bryck/prod-login.json
```

Removes an email address from the subscriber list.

Flow:

1. Log in to the Bryck REST API.
2. POST `/api/bcloud/notification_unsubscribe` with email.
3. Validate via GET `/api/bcloud/notification_subscribers`.
4. Display updated subscriber list.

Behaviour:

- `--email` is required.
- No state precondition — works in any Bryck lifecycle state.
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if email not provided or `login.json` is invalid.
- Exits 3 if validation failed.
- Formatted error display with unicode box and emoji.

5.21 `bryck_cloud_notification_list.py`

```
# Display notification configuration
python3 bryck_cloud_notification_list.py

# Save JSON to file
python3 bryck_cloud_notification_list.py --output config.json

# Custom credentials
python3 bryck_cloud_notification_list.py \
    --login /etc/bryck/prod-login.json
```

Displays the current notification configuration (if any).

Flow:

1. Log in to the Bryck REST API.
2. GET `/api/bcloud/notification_list`.
3. Display configuration in multi-line format.
4. Optionally write raw JSON to file if `--output` is specified.

Behaviour:

- Read-only operation — does not modify configuration.
- No state precondition — works in any Bryck lifecycle state.
- Display format: multi-line with 80-character dividers, aligned fields:
 - `CONFIG_ID`
 - `ENABLED` (true / false)
 - `CLOUD_TYPE` (SNS, SQS, etc.)
 - `SNS_TOPIC`
 - `SQS_QUEUE`
 - `STATES` (COMPLETED, FAILED, PAUSED)
 - `CONFIGURED_AT` (timestamp YYYY-MM-DD HH:MM:SS)
- Empty configuration prints “No notification configuration found” and exits 0.
- Optional JSON output via `--output PATH`.
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if login.json is invalid.
- Formatted error display with unicode box and emoji.

5.22 `bryck_cloud_notification_enable.py`

```
# Enable notifications
python3 bryck_cloud_notification_enable.py

# Custom credentials
```

```
python3 bryck_cloud_notification_enable.py \  
    --login /etc/bryck/prod-login.json
```

Activates notification delivery. Configuration must be setup first (§5.17).

Flow:

1. Log in to the Bryck REST API.
2. POST /api/bcloud/notification_enable.
3. Validate via GET /api/bcloud/notification_list until enabled=true (60s timeout).
4. Display configuration.

Behaviour:

- No parameters (besides --login).
- No state precondition — works in any Bryck lifecycle state.
- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if login.json is invalid.
- Exits 3 if validation times out after 60s.
- Formatted error display with unicode box and emoji.

5.23 bryck_cloud_notification_disable.py

```
# Disable notifications (preserve configuration)  
python3 bryck_cloud_notification_disable.py
```

```
# Custom credentials  
python3 bryck_cloud_notification_disable.py \  
    --login /etc/bryck/prod-login.json
```

Deactivates notification delivery while preserving configuration for later re-enabling.

Flow:

1. Log in to the Bryck REST API.
2. POST /api/bcloud/notification_disable.
3. Validate via GET /api/bcloud/notification_list until enabled=false (60s timeout).
4. Display configuration.

Behaviour:

- No parameters (besides --login).
- Configuration is **preserved** — use bryck_cloud_notification_delete.py (§5.24) to remove completely.
- No state precondition — works in any Bryck lifecycle state.

- Exits 0 on success.
- Exits 1 if API returns non-200 status.
- Exits 2 if login.json is invalid.
- Exits 3 if validation times out after 60s.
- Formatted error display with unicode box and 🐛 emoji.

5.24 bryck_cloud_notification_delete.py

```
# Delete configuration (with confirmation prompt)
python3 bryck_cloud_notification_delete.py

# Delete configuration (skip confirmation)
python3 bryck_cloud_notification_delete.py --force

# Custom credentials
python3 bryck_cloud_notification_delete.py \
    --login /etc/bryck/prod-login.json
```

Removes notification configuration entirely. **This action cannot be undone without reconfiguration.**

Flow:

1. Log in to the Bryck REST API.
2. **If not --force:** prompt user for confirmation.
3. If confirmed: POST /api/bcloud/notification_delete.
4. Validate via GET /api/bcloud/notification_list (config should be empty).
5. Display confirmation.

Behaviour:

- --force flag skips the confirmation prompt.
- No state precondition — works in any Bryck lifecycle state.
- On cancellation (user says “no”): exits 0 with message “Deletion cancelled by user”.
- On success: exits 0 and displays how to reconfigure via bryck_cloud_notification_setup.py.
- Exits 1 if API returns non-200 status.
- Exits 2 if login.json is invalid.
- Exits 3 if validation failed.
- Formatted error display with unicode box and 🐛 emoji.

6. Exit codes

Exit Code	Meaning	Action
0	Success	None — operation completed
1	HTTP/API error	Check error message, verify network
2	Invalid parameters or state	Review CLI arguments and login.json
3	Validation timeout	Increase timeout or check network
4	Transfer terminal failure	Review transfer report details
130	User interrupted (Ctrl+C)	User intentionally stopped
other	Unhandled exception	Check error logs

Exit Code Details:

- **Exit code 0:** All notification runners on success (§5.17-5.25); user cancels deletion (§5.24)
- **Exit code 1:** Used by `change_ip.py`, `change_time.py`, `bryck_info.py`, `bryck_network_info.py`, `bryck_report.py`, `bryck_cloud_transfer_status.py`, `bryck_cloud_transfer_report.py`, `bryck_cloud_notification_*.py` (all)
- **Exit code 2:** State precondition not met, bad parameters, or output directory issues. Used by `change_time.py`, `bryck_report.py`, `bryck_cloud_transfer_report.py`, `bryck_cloud_notification_*.py` (invalid parameters or missing login.json)
- **Exit code 3:** Validation timed out (ticker did not converge; `bryck_report.py` if capture didn't reach COMPLETED; `bryck_cloud_transfer_report.py` on filesystem write error or 0-byte response; `bryck_cloud_notification_*.py` if cross-validation fails)
- **Exit code 4:** Cloud transfer entered terminal-failure state (FAILED / STOPPED / CANCELLED) — `bryck_cloud_transfer.py` only

7. Key-file transfer detail

The format and mount runners do **not** invoke `mkdir` on the Bryck; the destination directory `/opt/bryck/bryckapi/downloads/` must already exist on the Bryck server (it does, on a stock install).

The transfer runs entirely over one paramiko `SSHClient` opened from the local machine to `bryckapi_host` (port 22, password auth using `bryckserver_username` / `bryckserver_password`). Two channels are used:

```
SFTP put    <local_key_file> -> /opt/bryck/bryckapi/downloads/keyfile
ssh exec    sudo -n chmod 0644 /opt/bryck/bryckapi/downloads/keyfile
```

`sudo -n` guarantees no interactive password prompt — configure passwordless sudo on the **Bryck** for the `bryckserver_username` Unix account so it may run `chmod 0644` on that fixed path. Example visudo line:

```
bryck ALL=(root) NOPASSWD: /usr/bin/chmod 0644 /opt/bryck/bryckapi/downloads/keyfile
```

Key files are tiny (typically <1 KB), so no explicit SFTP timeout is set. Host keys are auto-accepted on first use (see §9).

8. Expanded Troubleshooting Guide

8.1 Diagnosis by Symptom

Symptom: “Refusing to mount: Bryck is already in state 'Mounted'” **Cause:** Bryck is already mounted (you can't mount twice).

Fix:

```
# Unmount first  
python3 bryck_eject_unmount.py
```

```
# Then try mount again  
python3 bryck_mount.py
```

Symptom: “Refusing to eject: state='...' (must be 'Mounted')” **Cause:** Bryck is not mounted (nothing to eject).

Fix: Skip eject and proceed directly to next operation:

```
python3 bryck_format.py # If you need to reformat  
# or  
python3 bryck_remove.py # If you want to unregister from system
```

Symptom: “No logical cards reported by /api/config/info” **Cause:** API sees no attached storage (drives not detected).

Fix:

```
# 1. Physically verify the connection  
# 2. Run scan to force rediscovery  
python3 bryck_scan.py
```

```
# 3. Check again  
python3 bryck_info.py
```

Symptom: “bryck_format.py hangs on ‘Scanning...’” **Cause:** Drives not detected by Bryck firmware.

Fix:

```
# 1. Wait up to 5 minutes (formatting can be slow)  
# 2. If still hanging, interrupt and try scan  
Ctrl+C  
python3 bryck_scan.py  
  
# 3. Retry format  
python3 bryck_format.py
```

Symptom: “CloudTransfer returned state FAILED” **Cause:** Transfer failed (network, permissions, cloud issue, etc.).

Fix:

```
# 1. Get transfer details
python3 bryck_cloud_transfer_status.py --transfer-id 69

# 2. Check the report
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 69 --report-path .

# 3. Read transfer_report.json in the ZIP
unzip cloud_transfer_report_69.zip
cat transfer_report.json # See detailed error
```

Symptom: Connection timeout on cloud transfer **Cause:** Network connectivity issue or very slow upload.

Fix:

```
# 1. Check network connectivity to cloud
ping 8.8.8.8

# 2. Increase timeout in login.json
# Change "timeout": 300 to "timeout": 600

# 3. Retry with more time
python3 bryck_cloud_transfer_status.py --transfer-id 69
```

Symptom: “SSH connection refused (paramiko error)” **Cause:** SSH credentials wrong or SSH disabled on Bryck.

Fix:

```
# 1. Verify credentials in login.json
cat login.json | grep bryckserver

# 2. Test SSH directly
ssh bryck0192.168.6.35 "echo 'SSH works!'"

# 3. If that fails, SSH is not working
#   Contact Bryck admin to enable SSH or verify credentials
```

8.2 Error Codes Quick Reference

See §6 Exit codes for the complete reference table and detailed explanations of which runners use each code.

9. Performance & Tuning Guide

9.1 Timeout Configuration

All runners respect the `timeout` field in `login.json` (in seconds):

```
{  
  "timeout": 300 ← Default (5 minutes)  
}
```

When to adjust timeout:

Operation	Typical Duration	Suggested Timeout	When to increase
scan	30-60 seconds	120 s	Never (scan is fast)
format	300-600 seconds (5-10 min)	900 s (15 min)	If you have very large storage
mount	30-120 seconds	300 s (5 min)	Rarely needed
cloud transfer (validation)	2-30 seconds	120 s (2 min)	Usually not needed
status check	2-5 seconds	60 s	Never (instant API call)

Example: Increase timeout for large storage:

```
{  
  "timeout": 1800 ← 30 minutes for very large operations  
}
```

9.2 Expected Operation Durations

Rough estimates (depends on storage size and network):

Operation	Time	Varies by
bryck_scan.py	30-60 sec	Number of drives
bryck_format.py	5-15 minutes	Storage size (capacity $\times$ 1min per TB)
bryck_mount.py	30-120 sec	Storage size
bryck_eject_unmount.py	10-30 sec	—
bryck_erase.py	30-120 sec	—
Cloud transfer (start)	2-10 sec	Network latency to cloud
Cloud transfer (upload 1TB)	2-24 hours	Network speed (typical 100 Mbps = 12 hours)
Cloud transfer (download 1TB)	1-12 hours	Cloud download speed (usually faster than upload)

9.3 Improving Performance

Upload/download speed:

```
# Formula: Time = Data Size / Network Speed
# 1 TB at 100 Mbps = 8000 Mbps / 100 Mbps = 80,000 seconds = 22 hours

# To speed up:
# 1. Upgrade network link (use 1Gbps or faster)
# 2. Check cloud connection (some clouds are slow from certain networks)
# 3. Upload multiple transfers in parallel (cloud allows this)
```

Format/mount speed:

```
# Usually bottlenecked by storage hardware, not network
# 1 TB storage 1 minute to format
# Limited by: drive speed, RAID rebuild time, controller speed
```

10. Security & Credentials Best Practices

10.1 Credential Management

Where credentials are stored:

Credential	Location	Access	Sensitivity
login.json	Local file on operator machine	File permissions only	HIGH — Contains passwords
SSH keyfile	Local file (if encrypted)	File permissions only	HIGH — Encryption key
Cloud credentials	Inside cloud_ops.json	File permissions only	CRITICAL — Cloud access keys

Protect your credentials:

```
# 1. Restrict file permissions (owner read-only)
chmod 600 login.json
chmod 600 cloud_ops.json
chmod 600 format_mount_params.json

# 2. Don't commit to git
echo "login.json" >> .gitignore
echo "cloud_ops.json" >> .gitignore
echo "format_mount_params.json" >> .gitignore

# 3. Don't share over email or chat
# If you must share, rotate credentials immediately after

# 4. Use different credentials for dev/prod
# Keep separate login.json files:
# - login-prod.json (production Bryck)
# - login-dev.json (test Bryck)
# Use: python3 bryck_info.py --login login-prod.json
```

10.2 SSH Passwordless Sudo (required for format/mount/erase)

These runners need SSH sudo access to chmod keyfiles and perform operations.

Setup on Bryck (one time):

1. SSH to Bryck:

```
ssh bryck@192.168.6.35
```

2. Run visudo to edit sudoers:

```
sudo visudo
```

3. Add this line at the end (replace bryck with your SSH username if different):

```
bryck ALL=(root) NOPASSWD: /usr/bin/chmod 0644 /opt/bryck/bryckapi/downloads/keyfile
```

4. Save (Ctrl+X in nano, or :wq in vim)

5. Verify it works:

```
sudo -n chmod 0644 /opt/bryck/bryckapi/downloads/keyfile && echo "Sudo works!"
```

If you skip this: format/mount/erase will fail with SSH authentication errors.

10.3 Encryption Key Management

For encrypted Bryck (uses key_file in format__mount__params.json):

```
# 1. Generate encryption key (one time, keep it safe)
openssl rand -base64 32 > bryck.key
chmod 600 bryck.key

# 2. Reference it in format_mount_params.json
{
  "format": {
    "key_file": "/path/to/bryck.key",  ← file path for encrypted format
    "encryption_option": "",          ← always empty
    ...
  }
}

# 3. Backup the key file (not in git!)
# Store in: password manager, encrypted USB, HSM, or secure backup
cp bryck.key ~/encrypted-backups/bryck.key.backup

# 4. Mount uses same key
{
  "mount": {
    "key_file": "/path/to/bryck.key",  ← MUST match format's key
    "encryption_option": "",          ← always empty
    ...
  }
}

# WARNING: If you lose the key file, data is unrecoverable!
```

10.4 Cloud Credentials (AWS/GCP/Azure)

AWS (cloud_ops.json):

```
{
  "cloud_type": "aws",
  "access_key_id": "AKIAIOSFODNN7EXAMPLE",           ← Secret! Rotate regularly
  "secret_access_key": "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY", ← Secret! Rotate regularly
  "region": "us-west-1"
}
```

Best practices:

```
# 1. Use IAM user (not root account)
# 2. Grant minimum permissions (S3 bucket only, specific operations)
# 3. Rotate keys every 90 days
# 4. Use separate keys for dev/prod
# 5. Never commit to git
```

GCP (cloud_ops.json with keyfile):

```
{
  "cloud_type": "gcp",
  "keyfile": "/path/to/service-account.json"         ← Secret! Keep secure
}
```

Azure (cloud_ops.json):

```
{
  "cloud_type": "azure",
  "access_key_id": "storage_account_name",           ← Not super secret
  "secret_access_key": "DefaultEndpointsProtocol=https;AccountName=...", ← Secret!
  "tenant_id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx" ← Public OK
}
```

11. Examples Gallery — Real-World Scenarios

11.1 First-Time Setup Example

Goal: Format a new Bryck and mount it for use.

```

# 1. Create login.json
cp login.example.json login.json
nano login.json
# (Fill in: host=192.168.6.35, username=admin, password=...)

# 2. Test connectivity
python3 bryck_info.py
# Output: State is " Removed"

# 3. Scan
python3 bryck_scan.py
# Output: Scan complete (4 drives detected)

# 4. Format (using defaults from format_mount_params.json)
python3 bryck_format.py
# Output: Bryck formatted (RAID 5, ZFS, 4.0 TB)
# (Takes ~5-10 minutes depending on storage size)

# 5. Mount
python3 bryck_mount.py
# Output: Bryck mounted at /bryck

# 6. Verify
df -h /bryck
# Output: /bryck 4.0T 0 4.0T 0% /bryck

```

11.2 Cloud Transfer to AWS Example

Goal: Upload data from Bryck to AWS S3.

Preparation:

```

# 1. Create cloud_ops.json (or edit existing)
{
  "cloud_type": "aws",
  "access_key_id": "AKIA...",
  "secret_access_key": "wJal...",
  "region": "us-west-1",
  "bryck_src": "/bryck/data",           ← Source folder on Bryck
  "cloud_bucket": "s3://my-bucket/upload", ← S3 destination
  "bryck_dst": "/bryck/download"       ← (Not used for upload)
}

```

Execution:

```

# 1. Configure AWS (one time)
python3 bryck_cloud_configure.py

```

```

# Output:  AWS configured

# 2. Start upload
python3 bryck_cloud_transfer_initiate.py --mode upload
# Output:  Transfer started (ID: 69)

# 3. Monitor progress
python3 bryck_cloud_transfer_status.py --transfer-id 69
# Output:  TRANSFER_ID=69 : STATE=IN_PROGRESS : PROGRESS=500 GB / 2000 GB (25% completed)

# 4. Wait for completion (check periodically)
while true; do
    python3 bryck_cloud_transfer_status.py --transfer-id 69
    sleep 60 # Check every minute
done

# 5. Get report
python3 bryck_cloud_transfer_report.py --cloud-transfer-id 69 --report-path .
# Output:  cloud_transfer_report_69.zip
unzip cloud_transfer_report_69.zip
cat transfer_summary.txt

```

11.3 Setup Email Notifications Example

Goal: Get notified when cloud transfers complete.

Preparation:

```

# Create AWS SNS topic (in AWS console):
# arn:aws:sns:us-west-1:123456789012:bryck-notifications

# Update cloud_ops.json:
{
    "notification": {
        "sns_topic": "arn:aws:sns:us-west-1:123456789012:bryck-notifications",
        "emails": ["admin@company.com", "ops@company.com"],
        "states": ["COMPLETED", "FAILED"]
    }
}

```

Execution:

```

# 1. Setup notifications
python3 bryck_cloud_notification_setup.py
# Output:  Notification setup complete

# 2. Subscribe emails (if not already in setup)

```

```
python3 bryck_cloud_notification_subscribe.py --email manager@company.com  
# Output: Email subscribed
```

3. View subscribers

```
python3 bryck_cloud_notification_subscriber_show.py  
# Output: Lists all subscribers
```

4. Enable notifications

```
python3 bryck_cloud_notification_enable.py  
# Output: Notifications enabled
```

Now when transfers complete, subscribers get email notifications!

11.4 Disaster Recovery: Wipe and Reformat

Goal: Completely erase data and reformat Bryck.

WARNING: This destroys all data on the Bryck!

1. Unmount

```
python3 bryck_eject_unmount.py  
# Output: Bryck ejected
```

2. Erase (wipes data)

```
python3 bryck_erase.py  
# Output: Store cleared
```

3. Rediscover drives

```
python3 bryck_scan.py  
# Output: Scan complete
```

4. Reformat

```
python3 bryck_format.py  
# Output: Bryck formatted
```

5. Remount

```
python3 bryck_mount.py  
# Output: Bryck mounted
```

Done! Bryck is fresh and ready to use

12. Technical Specifications

12.1 Software Requirements

Operator Machine:

Component	Version	Notes
Python	3.7+	3.10+ recommended for best performance
requests	2.25+	HTTP library for REST API calls
paramiko	2.7+	SSH library for key file transfers
OpenSSL	1.1+	For HTTPS support

Installation verification:

```
python3 -c "import requests, paramiko; print(' All packages installed')"
```

Bryck Appliance:

Component	Version	Notes
Firmware	1.0+	All runners work with v1.0+, v1.2+ recommended
REST API	Enabled	Must be running (default on all Bryck models)
SSH	Enabled	Required for format/mount/erase/scan
NFS/SMB	Optional	Not required for these runners

12.2 Network Requirements

Connectivity: - **REST API:** TCP port 80 (HTTP) or 443 (HTTPS) - **SSH:** TCP port 22 (required for format/mount/erase/scan) - **Cloud transfers:** Outbound access to cloud provider (AWS/GCP/Azure)

Bandwidth: - Minimal for API calls (<1 Mbps) - For cloud transfers: recommend 100 Mbps+ for fast uploads

Firewall rules (minimum):

Operator machine → Bryck

- TCP port 80 (REST API)
- TCP port 22 (SSH)

Bryck → Cloud provider (for transfers)

- Outbound TCP 443 (HTTPS to AWS/GCP/Azure)

12.3 Storage Performance Characteristics

Typical throughput by operation:

Operation	Throughput	Notes
Format	~1 min per TB	Depends on drive speed & controller
Mount	~30 sec	Filesystem check (if needed) adds time
Erase	~1 min per TB	Data sanitization is slow
Cloud upload	Network limited	Typical 10-100 Mbps depending on ISP
Cloud download	Cloud limited	Usually faster than upload (100-500 Mbps)

Estimating cloud transfer time:

Time (hours) = Data Size (GB) / (Network Speed (Mbps) / 8)

Example: 1000 GB at 100 Mbps
= 1000 / (100/8) = 1000 / 12.5 = 80 hours

Example: 1000 GB at 1000 Mbps (1 Gbps)
= 1000 / (1000/8) = 1000 / 125 = 8 hours

12.4 API Endpoints Reference

All REST endpoints used by runners:

Core Lifecycle & Configuration

Endpoint	Method	Purpose
/api/config/info	GET	Get Bryck state & info
/api/config/scan	POST	Discover drives
/api/config/configure	POST	Configure filesystem
/api/config/mount	POST	Mount filesystem
/api/config/eject	POST	Unmount & eject
/api/config/reset_store	POST	Erase data
/api/config/remove	POST	Unregister Bryck
/api/network/configure	POST	Set IP address
/api/settings/set_date	POST	Set system time

/api/download | GET | Download files |
/api/tasks/list | GET | Query task status |

Cloud Provider & Transfer

Endpoint	Method	Purpose
/api/bcloud/config	POST	Configure provider
/api/bcloud/config_list	GET	List configs
/api/bcloud/config_remove	POST	Remove config
/api/bcloud/transfer	POST	Start transfer
/api/bcloud/pause_transfer	POST	Pause transfer

Endpoint	Method	Purpose
/api/bcloud/resume_transfer	POST	Resume transfer
/api/bcloud/cancel_transfer	POST	Cancel transfer
/api/bcloud/status_transfer	GET	Get status
/api/bcloud/list_transfer	POST	List transfers

Notifications

Endpoint	Method	Purpose
/api/bcloud/notification_setup	POST	Setup notifications
/api/bcloud/notification_list	GET	Get config
/api/bcloud/notification_subscribe	POST	Subscribe email
/api/bcloud/notification_unsubscribe	POST	Unsubscribe email
/api/bcloud/notification_subscribers	GET	List subscribers
/api/bcloud/notification_enable	POST	Enable
/api/bcloud/notification_disable	POST	Disable
/api/bcloud/notification_delete	POST	Delete config

Response format (all endpoints):

```
{
  "error": null,           ← Null if successful, error object if failed
  "result": {...}         ← Operation result (varies by endpoint)
}
```

Error response example:

```
{
  "error": {
    "message": "Bryck is already in state ' Mounted'",
    "code": "STATE_ERROR"
  },
  "result": null
}
```

12.5 Common Error Messages

- **sudo: a password is required** — configure passwordless sudo on the Bryck for bryckserver_username (see §7 for the visudo line).
- **bryckapi_host must be an IPv4 address** — replace localhost / hostnames in login.json with the Bryck's IPv4 address.
- **Correcting port 80 -> 443 for scheme=https** (INFO log) — informational only. bryckapi_port was auto-corrected to match bryckapi_scheme; supply matching values to silence it.

- `paramiko.ssh_exception.AuthenticationException` — verify `bryckserver_username` / `bryckserver_password` and that `sshd` on the Bryck accepts password auth (`PasswordAuthentication` yes).
- `SshRunnerError: SSH connect to ... failed` — SSH (port 22) is unreachable from the machine running the runner. Check firewalls and that `sshd` is running on the Bryck.
- **Login failures / connection reset** — check `login.json` (`bryckapi_host`, `bryckapi_port`, `bryckapi_scheme`) and that the API service is running.
- **Timeouts** — the polling helpers use fixed budgets (`SCAN_TIMEOUT`, `CONFIGURE_TIMEOUT`, `EJECT_TIMEOUT`, `ERASE_TIMEOUT`, `REMOVE_TIMEOUT`, `CHANGE_IP_TIMEOUT`, `CHANGE_TIME_TIMEOUT`). Increase in the runner source if your hardware needs longer.
- **`change_ip.py` validation timeout when changing the management interface** — the client's own session drops mid-configure so `bryck_info()` cannot be polled. The change still lands; verify manually via a session on the new IP.
- **`change_time.py` Manual validation drift** — `server_time` is compared with a ± 120 s tolerance (`TIME_TOLERANCE_SECONDS`). If your input timezone differs from the Bryck server timezone (`server_info.server_timezone`), bump the tolerance or set the JSON date/time in the server timezone.
- **`change_time.py` shows 401 / 422 immediately after `set_date`** — the JWT was issued before the clock jump, so its `iat` is now either too old (401 “Signature has expired”) or in the future (422 “Token is not yet valid”). `change_time.py` re-logs in automatically before polling; if you see this error, the deployed script is out of date — redeploy the current `change_time.py`.
- **`bryck_cloud_transfer.py`: Bad cloud parameters: `cloud_type=...` requires field(s) [...]** — inspect `cloud_ops.json` and cross-check it against the per-cloud requirements matrix in §2.5.
- **`bryck_cloud_transfer.py`: GCP keyfile does not exist on this machine** — keyfile must be a path readable on the machine running the runner (it is SFTP-uploaded to the Bryck).
- **`bryck_cloud_transfer.py`: GCP keyfile placement failed: ... sudo: a password is required** — configure passwordless sudo on the Bryck for `bryckserver_username` on `mkdir -p`, `mv`, and `chmod 0644` targeting `/opt/bryck/bryckapi/downloads/deployment/.gcloud/`.
- **`bryck_cloud_transfer.py`: transfer stayed at `IN_PROGRESS` but my data never arrived** — the runner intentionally does not poll for `COMPLETED`. Copy the highlighted transfer IDs from its success banner and run `python3 bryck_cloud_transfer_status.py --transfer-id <id>` (§5.15) — or the same command with no `--transfer-id` to see every active transfer — to confirm the final state.
- **`bryck_cloud_transfer.py`: Upload transfer <id> did not reach `IN_PROGRESS` in 120s** — the server accepted the transfer request but never advanced its state. Bump `TRANSFER_START_TIMEOUT` in `bryck_cloud_transfer.py`, or inspect the transfer queue via `bryck_cloud_transfer_status.py` (no `--transfer-id`) to find the stalled job.
- **`bryck_cloud_transfer.py`: Refusing to start cloud transfer: `state='...` (must be 'Mounted')** — the Bryck is not mounted. Run `bryck_mount.py` first (or `bryck_scan.py` → `bryck_format.py` → `bryck_mount.py` on a fresh Bryck), then re-run the transfer.
- **`bryck_cloud_deconfigure.py`: Cloud deconfiguration validation FAILED after 60s (<type> still present in `config_list`)** — the `config_remove` POST returned OK but the entry did not disappear from `config_list`. Confirm the exact `cloud_type` was previously configured (case-insensitive match on `bcloud_type` / `cloud_type` fields in

`config_list`), then retry.

- **bryck_cloud_deconfigure.py:** `argparse invalid choice: '...'` — `--cloud-type` must be one of `aws`, `gcp`, `azure`.
 - **bryck_cloud_transfer_status.py:** `Transfer <id> not found on the Bryck` — the ID does not exist on this Bryck (never created, or purged after `COMPLETED`). Run the runner with no `--transfer-id` to see the current transfer list.
 - **bryck_cloud_transfer_status.py:** `No active cloud transfers found` — every transfer known to the Bryck is in `COMPLETED` state. If you expected an in-flight transfer, re-check the transfer ID that `bryck_cloud_transfer.py` printed in its success banner.
 - **bryck_cloud_transfer_report.py:** `TypeError: ApiSession.get() got an unexpected keyword argument 'stream'` — `session.py` is out of date. Redeploy the current version, which accepts `stream=True` on `ApiSession.get()`.
 - **bryck_cloud_transfer_report.py:** `Report for transfer <id> downloaded 0 bytes` — the Bryck accepted the request but sent an empty body. Verify the transfer ID exists via `bryck_cloud_transfer_status.py --transfer-id <id>` and that at least one state transition has occurred (fresh transfers may not have a report yet).
 - **bryck_cloud_transfer_report.py:** `Destination directory does not exist` — `--report-path` was a full file path whose parent directory is missing. Create the directory or pass an existing one as `--report-path`.
-

9. SSH transport (`ssh_runner.py`)

All runners that need to inspect / mutate the Bryck's OS shell (`bryck_scan.py`, `bryck_format.py`, `bryck_mount.py`, `bryck_erase.py`) do so through `ssh_runner.SshRunner` — a thin pure-Python wrapper around a single `paramiko.SSHClient`.

Key properties:

- **Transport** — one persistent SSH transport per runner invocation. `run()` and `put()` open fresh channels on that shared transport, so each call skips re-authentication.
- **Auth** — password only, via `bryckserver_username` and `bryckserver_password` from `login.json`. Public-key auth and SSH agents are **disabled** on the client side to keep behaviour deterministic (`allow_agent=False`, `look_for_keys=False`).
- **Host key policy** — `paramiko.AutoAddPolicy`. Unknown host keys are silently trusted and appended to the runner user's `~/.ssh/known_hosts` on first use. For strict deployments, switch to `RejectPolicy` and pre-populate `known_hosts`.
- **Port** — hardcoded to 22. Change in `ssh_runner.DEFAULT_SSH_PORT` if your Bryck listens elsewhere.
- **Command execution** — `run(cmd)` accepts a single string that the remote shell interprets. Callers must `shlex.quote()` any untrusted input. Today only static command literals are used, so no injection risk.

Because the runner opens the SSH connection **from** the machine executing the script, it is now fully platform-independent — the same codebase works on the Bryck itself or on any workstation that can reach the Bryck's REST port and port 22.

Dependencies — `paramiko >= 3.4.0` (declared in the project-level `pyproject.toml`; install with `pip install paramiko` on hosts that lack it).